New Generation Data Models and DBMSs Course - Assignment 2 - Crowdfunding Company

Carlo Falchi (Matr. 67340A)[1*]
and Francesco Palladini (Matr. 85652A)[1*]

[1]Department of Computer Science, University of Milan, Italy.

*Corresponding author(s). E-mail(s): carlo.falchi@studenti.unimi.it;
francesco.palladini@studenti.unimi.it;

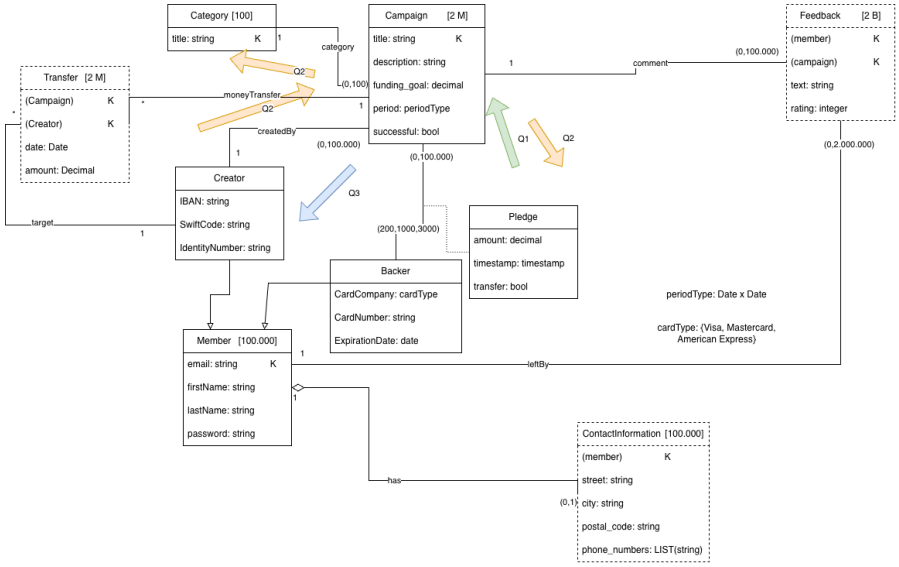


Fig. 1: Conceptual Diagram

1 Solution

The class Member has been chosen as a generalization of the classes Creator and Backer, because this simplifies the identification of a member, and it is also useful to implement feedback and contact information. Each member is associated with zero or one Contact information. Both the Creator and Backer classes are associated with the class Campaign. A campaign can be created by only one creator. A backer can promise pledges, modeled by the association Pledges, to a number of campaigns between 200 and 3000, with an average of 1000, as specified in the text. The attributes of Pledge are the amount, timestamp, and transfer, a boolean attribute which determines if the promised amount is transferred. Pledge is connected to the Transfer class, a class which represents the transfer of money, by the association transferredTo: multiple pledges can be transferred to a transfer, but each pledge can be associated with only one transfer. The class Category associated with Campaign has the purpose of answering the second query, which otherwise could not be executed. A campaign can be associated with only one category, while a category can be associated with multiple campaigns. Finally, we have the class Feedback, which is a weak class identified by Member and Campaign. This class has a maximum of 2 billion instances, because we have annually 100,000 users and 2 million campaigns, so the multiplication of the number of users times the number of campaigns gives 2 billion total possible feedbacks per year. Of course, 2 million feedbacks can be left by a user, while a user can be associated only with their feedbacks, and each campaign can receive 100,000 feedbacks, which is also the number of users, while a campaign is associated only with its feedbacks.

In the diagram we have considered the following structured types:

- cardType: {Visa, Mastercard, American Express}
- periodType: Date x Date

1.1 Assumptions

1. First, we assume that the conceptual class diagram reports the annual number of instances for classes and relations.
2. Since the number of categories is not given, we assume that there are 100, which is a reasonable number of categories.
3. As the campaign period ends, and the system determines whether the campaign is successful, the attribute successful of the class Campaign is set to True.
4. The attribute successful of Campaign has a default value of False.
5. We assume the user will log in to the platform using an email and a password, so we set email as the key of the Member class.
6. We assume that the same backer can make multiple pledges to the same campaign, with each pledge recorded separately along with its amount and timestamp.
7. Feedback may be submitted only for successful campaigns. The campaign creator and each participating backer may submit at most one feedback item per campaign.
8. We assume that categories can change, and new categories can be added over time.

1.2 Constraints

The following constraints can be identified in the considered domain. Note that we report only the constraints that are not already included in the representation of the class diagram:

1. The password field of the Member class is an encrypted field.
2. A campaign is successful if the total amount of pledges meets or exceeds the funding goal.
3. A pledge and its timestamp should be created only within the date range (period) of the campaign.
4. The IBAN of the creator should have a length between 15 and 34 characters and comply with the ISO 13616 standard.
5. The SwiftCode should follow the ISO 9362 standard and be between 8 and 11 characters.
6. When the campaign ends and is successful, the pledged funds are transferred from the backers to the creator.
7. The date of the transfer should be after the end (period) of the campaign.
8. Creators and Backers can leave feedback only after the campaign ends successfully.
9. Feedback consists of a numerical rating from 1 to 10.
10. The end date of the campaign should be a date later than the start date.
11. The attribute amount in Transfer is the sum of the amounts of the pledges involved in the transfer.

1.3 Workload

The workload is shown in Figure 1.

1. To identify the top 5 most competitive campaigns, the entry point is the class Pledge. Pledges are grouped by Campaign and ordered by the total number of pledges, and then we navigate to Campaign to get the title of the campaign. So, the query path is (Pledge)→(Campaign).
2. The entry point is the Creator class, then we identify the category through Campaign and get the highest average pledge amount in the Pledge class. The query path is (Creator) $\underset{creator}{\rightarrow}$ (Campaign)→(Pledge) and Campaign $\underset{category}{\rightarrow}$ Category to identify the titles of the categories.
3. To identify the top 10 creators per month in the last year by number of successful campaigns, the entry point is the class Campaign. Campaigns are filtered by successful and end_date within the last year, and then grouped by the month of end_date and by their creator, navigating to the Creator class to order and select the top 10 creators for each month. So, the query path is (Campaign) $\underset{createdBy}{\rightarrow}$ (Creator).

1.4 Logical Model

Figure 1 shows the navigational paths for the three workloads:

- Q1: (Pledge)→(Campaign)

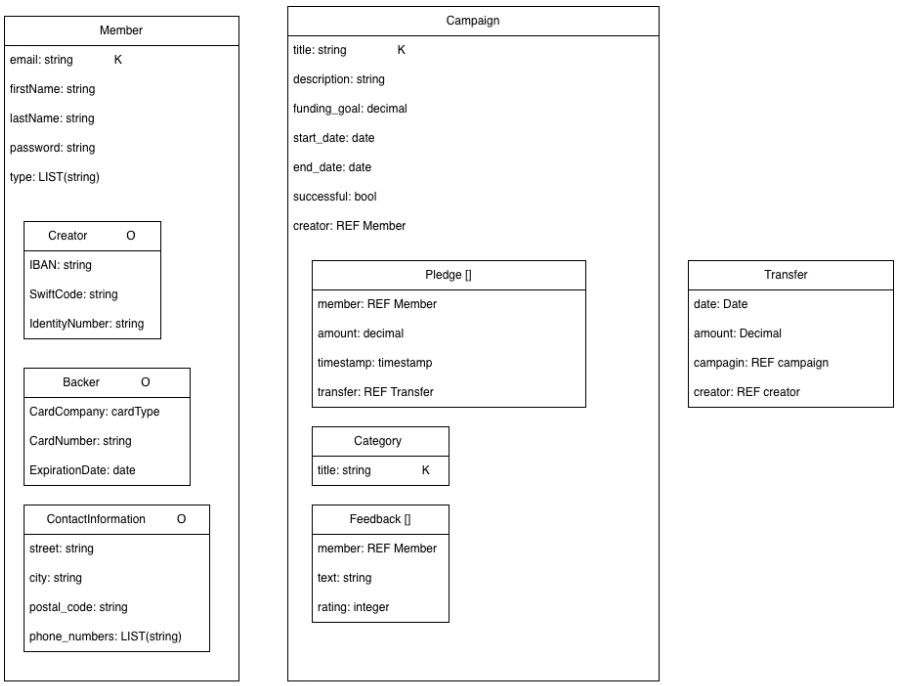


Fig. 2: Logical Model

- Q2: (Creator) $\underset{creator}{\rightarrow}$ (Campaign)→(Pledge) and Campaign $\underset{category}{\rightarrow}$ Category to identify the titles of categories
- Q3: (Campaign) $\underset{createdBy}{\rightarrow}$ (Creator)

Figure 2 reports the logical model that we have designed, taking into account the above operations. The first operation uses Pledge as an entry point, the second uses Creator, and the third uses Campaign. Q1 and Q2 are in conflict because they have different directions; however, Campaign is used in all the workloads. Furthermore, given that a Pledge can be assigned to only one Campaign, we suggest using Campaign as an entry point and thus as a collection, embedding Pledge in it without risk of duplication. To improve the performance of the second operation (Campaign → Category), we embedded Category inside Campaign so we can avoid the lookup operation; it can introduce duplication, but it is considered acceptable. Regarding the third operation (Campaign → Creator), which uses Campaign as an entry point, it confirms the need to use Campaign as a collection and Creator as a reference using a lookup to avoid duplication of Creators. The Feedback collection has been embedded within Campaign, given that there is a one-to-many relationship between Campaign and Feedback, and to avoid duplication, we use a reference to the Member class. The Transfer collection has been created as a separate collection because it is not included in the most important workloads. We expect that a report or a page will request all transfers rarely, so we can insert Campaign and Creator as references and use the

lookup operation for them. Finally, the Member collection has been created as an entry point and as a separate collection, embedding in it its specializations Creator and Backer. Optional contact information has been embedded in it, but there are further improvements explained in the next section.

1.5 Design Patterns

First, in order to save computations, we can apply the computed pattern to the attribute amount in Transfer. We can use the extended reference pattern by duplicating the attributes firstName and lastName of Creator in the Campaign class, to make the execution of queries 2 and 3 faster. We can also apply the subset pattern by creating a separate collection for ContactInformation, which is not involved in the workload, and so we expect that it is accessed less often.

1.6 Bonus Workload

We identified a different workload, which is the following:

1. Identify the top 10 successful campaigns that reached the highest transfer amount.
2. Identify the top 10 cities with the highest number of backers who promised a pledge.
3. For each member, identify the number of feedbacks he/she left.

This workload needs a different logical model, as explained in the following:

- For query 1, we should embed Transfer into Campaign to make the query efficient, because otherwise we would have to do a lookup operation for all the successful campaigns.
- For query 2, we should embed Pledge into Backer instead of Campaign, because in this way we avoid doing the lookup operation (the backers are potentially 100,000, so it would be costly).
- For query 3, we should embed Feedback in Member instead of Campaign, because otherwise we would need to do a lookup, and it is also easier to access a maximum of 100,000 documents instead of 2 million.